PhysicsLM: Autoregressive Language Modeling of 2D Rigid Body Dynamics

Anonymous Authors[1]

Abstract

We present **PhysicsLM**, a system that frames 2D rigid body physics simulation as autoregressive language modeling. Rather than constructing a bespoke neural physics engine, we encode each simulation frame as a short, structured text string and fine-tune a 350M-parameter language model (LFM2-350M) to predict the next frame token-by-token using standard next-token prediction. To support training and evaluation, we introduce **PhysicsScenes**, the largest text-format 2D physics dataset to date, comprising 900K training scenes across 24 scenario types drawn from six physical categories (collisions, stacking, ramps, constraints, minigames, and complex compound scenes), plus a held-out out-of-distribution (OOD) split of six unseen scenario types. On single-step prediction, our fine-tuned model achieves a position RMSE of **18.85 px** (MSE 355 px^2) on seen scenario types and a **0.0% parse failure rate** across all 30 scenario types. OOD results split by novelty: near-distribution OOD types (bowling, ramp_roll, newtons_cradle) achieve **0.94 px** RMSE, while genuinely novel scenarios (angry_birds, hourglass, pong) reach **24.79 px** RMSE—a $1.3\times$ degradation relative to the seen-scenario average. Performance spans five orders of magnitude across categories: near-perfect on stacking and constraint scenes (RMSE < 1.5 px) and substantially worse on high-entropy minigame and complex scenarios. We additionally demonstrate first-of-its-kind in-browser LM physics simulation via ONNX/WebGPU export, eliminating the need for server-side inference.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

1. Introduction

Physics simulation is central to robotics, game engines, scientific computing, and increasingly to reinforcement learning environments. Classical solvers based on constraint-satisfaction (e.g., Chipmunk2D, Bullet, MuJoCo) are both accurate and fast, but they are not differentiable, require hand-specified object geometries, and cannot seamlessly incorporate learned priors about typical physical dynamics.

The rise of *neural physics simulators*—systems that learn to approximate physical dynamics from data—has opened a complementary research direction. Methods based on graph neural networks (GNNs) (Battaglia et al., 2016; Sanchez-Gonzalez et al., 2020; Li et al., 2019) can generalize across scenes and object counts, and recent mesh-based variants (Pfaff et al., 2021) operate on complex geometries. Yet these specialized architectures require curated graph construction, custom aggregation schemes, and careful architecture decisions for each new domain.

Meanwhile, large language models (LLMs) have shown remarkable capabilities for structured prediction across diverse domains. Text-based representations have proven effective for tasks as varied as code generation, molecular property prediction, and robot planning (Wei et al., 2022; Yang et al., 2023). This raises a natural question: *can a language model learn to simulate physics through text?*

In this paper we answer affirmatively. We introduce **PhysicsLM**, a system that represents each simulation frame as a human-readable text string of the form:

```
Frame 42: Objects in motion.
  obj_0: pos=(312.48, 187.23),
         vel=(45.21, -123.45)
  obj_1: pos=(201.12, 98.77),
         vel=(-12.34, 34.56),
         a=0.87, av=1.23
```

and trains LFM2-350M (Liquid AI, 2024) via LoRA fine-tuning (Hu et al., 2022) to predict the next such frame. Physics rollout then becomes *pure language model generation*—no symbolic solver is invoked at test time.

Our contributions are:

1. **PhysicsScenes dataset**: 900K 2D rigid-body scenes spanning 30 scenario types in six physical categories,

generated with Pymunk (Blomqvist, 2024) at 60 FPS. The dataset includes a designed OOD split to probe generalization.

2. **Text-as-physics representation**: We show that decimal text encoding of position/velocity/angle suffices for a pre-trained LM to learn physically plausible dynamics, eliminating the need for bespoke tokenizers or graph structures.
3. **Curriculum training**: We assign difficulty levels 1–5 to scenarios based on object count and physical complexity and employ difficulty-ordered curriculum training throughout; formal ablation against uniform sampling is deferred to future work.
4. **WebGPU browser deployment**: We export the fine-tuned model to ONNX with q4f16 quantization and run it in-browser via WebGPU, achieving the first in-browser LM physics simulator without server-side inference.

2. Related Work

Neural Physics Simulators. Learning-based physics engines date back to Interaction Networks (Battaglia et al., 2016), which represented scenes as graphs and applied GNNs to predict state transitions. DPI-Nets (Li et al., 2019) extended this to particle-based deformable and rigid objects. Learning to Simulate (GNS) (Sanchez-Gonzalez et al., 2020) demonstrated that GNNs can generalize across particle counts and materials for fluids, sand, and rigid bodies. MeshGraphNets (Pfaff et al., 2021) further scaled GNS-style simulation to mesh-based representations suitable for finite-element problems. Neural Relational Inference (Kipf et al., 2018) jointly infers interaction graphs and dynamics. In contrast to all of these, PhysicsLM requires no graph construction, no custom aggregation, and no physics-specific architecture.

Physics-Aware World Models. World models (Ha & Schmidhuber, 2018) and latent-space imagination methods (Hafner et al., 2020) learn to predict future observations in learned latent spaces. These approaches operate in pixel or compact latent spaces and are typically tied to specific visual environments. Decision Transformer (Chen et al., 2021) demonstrated that transformers can model sequential decision-making, motivating our use of autoregressive transformers for sequential physics prediction.

Physics-Informed Inductive Biases. Hamiltonian Neural Networks (Greydanus et al., 2019) and Lagrangian Neural Networks (Cranmer et al., 2020) embed energy conservation into the network structure. While elegant, these methods require structured low-dimensional state spaces and do not easily extend to scenes with many heterogeneous objects. PhysicsLM learns conservation laws implicitly through data.

Equivariant and Operator-Based Simulators. EGNN (Satorras et al., 2021) and SE(3)-equivariant variants enforce geometric symmetries (rotation/reflection invariance) by construction, achieving strong data efficiency on molecular dynamics. PAINET and similar architectures extend this to rigid-body multi-object settings with explicit pairwise interactions. Fourier Neural Operators (FNO (Li et al., 2021)) and state-space model (SSM) based operators learn solution operators for PDEs directly in frequency space, enabling resolution-independent generalization. These approaches demonstrate important trade-offs between structural inductive bias and architectural flexibility; PhysicsLM sits at the opposite extreme, sacrificing structured bias entirely in favor of a general-purpose LM backbone. A direct comparison with equivariant baselines on the PhysicsScenes benchmark is a natural extension of this work.

LLMs for Structured Prediction. GPT-3 (Brown et al., 2020) and subsequent LLMs (Touvron et al., 2023; OpenAI, 2023) have demonstrated that pre-trained transformers can be adapted to arbitrary structured-text tasks via few-shot prompting or fine-tuning. Chain-of-thought prompting (Wei et al., 2022) has shown that step-by-step text reasoning improves quantitative prediction. Recent work (Yang et al., 2024) benchmarks LLM qualitative physics reasoning, but does not train LLMs for precise quantitative state prediction.

3. Problem Formulation

State Representation. A simulation scene $\mathcal{S}$ consists of a static header $\mathbf{h}$ (gravity vector, timestep Δt, static geometry, initial object properties) and a sequence of frames $s_1, s_2, \ldots, s_T$. Each frame s_t specifies the state of all N dynamic objects at time step t:

$$s_t = \left\{(p_t^i,\, v_t^i,\, \theta_t^i,\, \omega_t^i)\right\}_{i=1}^{N}, \tag{1}$$

where $p_t^i \in \mathbb{R}^2$ is position, $v_t^i \in \mathbb{R}^2$ is linear velocity, $\theta_t^i \in \mathbb{R}$ is angle (radians), and $\omega_t^i \in \mathbb{R}$ is angular velocity.

Text Encoding. Each object's state is serialized as a single text line with 4-decimal-place precision. Angles and angular velocities are omitted for purely translational objects (e.g., circles with no rotation tracking). The header encodes gravity, timestep, and static geometry as a human-readable preamble. A special prompt token `Predict next frame:` is appended before generation.

Prediction Objective. Given the scene header $\mathbf{h}$ and a context window of the last K frames $s_{t-K+1:t}$, the model predicts the next frame as a text string:

$$p(s_{t+1} \mid s_{t-K+1:t}, \mathbf{h}) = \prod_{j=1}^{|s_{t+1}|} p_\theta(\text{tok}_j \mid \text{tok}_{<j}, s_{t-K+1:t}, \mathbf{h}), \tag{2}$$

where tok_j are the tokens of the serialized frame string and θ denotes model parameters. This is trained with standard cross-entropy loss over token sequences. At inference time we decode autoregressively frame by frame, feeding each predicted frame back as context for the next (rollout mode) or using ground-truth context (teacher-forced single-step mode).

Rollout Protocol. For multi-step rollout we maintain a sliding window of the last $K = 4$ predicted frames in the prompt. When the prompt exceeds 8192 tokens, we trim the oldest frames while always retaining the static header. This keeps inference cost bounded and prevents repetitive degeneration.

4. Dataset: PhysicsScenes

4.1. Scenario Taxonomy

We design 30 scenario types organized into six physical categories, as summarized in Table 1. Each category exercises a distinct aspect of 2D rigid-body dynamics: momentum transfer, gravitational stacking, inclined-plane motion, joint constraints, interactive minigames, and multi-phenomenon compound scenes. We use Pymunk (Blomqvist, 2024), a Python binding for the Chipmunk2D physics engine, as our simulation backend.

Table 1. PhysicsScenes scenario taxonomy. OOD indicates held-out types not seen during training. Difficulty is the curriculum label (1=easy, 5=hard).

Category	Scenario	Diff.	OOD
Collision	billiards	2	
	bowling	2	✓
	head_on	1	
	explosion	3	
	projectile	2	
Stacking	pyramid	2	
	tower	2	
	jenga	3	
	dominos	3	
	bridge	4	
Ramp	ramp_roll	1	✓
	ski_jump	2	
	marble_run	3	
	avalanche	3	
	plinko	4	
Constraint	pendulum	2	
	chain	3	
	seesaw	2	
	wrecking_ball	3	
	orbit	4	
Minigame	basketball	3	
	conveyor	3	
	pong	2	✓
	wind	3	
	breakout	3	
Complex	angry_birds	4	✓
	hourglass	5	✓
	newtons_cradle	4	✓
	rube_goldberg	5	
	pinball	4	

4.2. Generation Pipeline

Scenes are generated procedurally by sampling random seeds to vary object counts, initial positions, velocities, and material properties (friction μ, restitution e). Each scene is simulated for $T = 200$ frames at 60 FPS ($\Delta t = 1/60\,\text{s}$) using Pymunk. The full simulation runs on CPU (Pymunk has no GPU backend); we parallelize generation across 22 CPU cores, completing the 1M-scene generation run in approximately 29 minutes.

Each scene is serialized to a JSONL file: one header line followed by 200 frame lines. The text representation of a frame averages ≈120 tokens for a 5-object scene, yielding roughly 25K tokens per full scene. Dataset statistics are given in Table 2.

4.3. OOD Split Design

We designate six scenario types as held-out: three "simple OOD" types (*bowling*, *ramp_roll*, *pong*) that involve familiar physics but with novel spatial configurations, and three "complex OOD" types (*angry_birds*, *hourglass*, *newtons_cradle*) that combine multiple physical phenomena simultaneously. These types are entirely absent from training, allowing us to measure systematic generalization rather than interpolation within the training distribution.

4.4. Difficulty Curriculum Labels

Each training scene is assigned a difficulty label $d \in \{1, 2, 3, 4, 5\}$ based on (a) the scenario's physical complexity, (b) object count, and (c) presence of joint constraints. Difficulty 1 corresponds to single-object projectiles or head-on collisions; difficulty 5 corresponds to hourglass or Rube Goldberg machines with 10+ objects and multi-stage interactions. These labels drive the curriculum training schedule described in Section 5.

Table 2. PhysicsScenes dataset statistics.

Split	Scenes	Scenario Types
Train	900,000	24 (seen)
Validation	100,020	30 (all)
OOD subset	20,004	6 (held-out)
Total	1,000,020	30
Scene properties		
Objects per scene	2–15 (avg ≈5)	
Frames per scene	200	
Simulation FPS	60	
Canvas size	800×600 px	
Uncompressed size	≈582 GB	

5. Method

5.1. Base Model: LFM2-350M

We use LFM2-350M (Liquid AI, 2024), a compact language model from Liquid AI designed for efficient edge deployment, as our pre-trained backbone. LFM2-350M is a transformer with a context window of 8192 tokens and a vocabulary tuned for multilingual and code-like structured text. We apply parameter-efficient fine-tuning via LoRA (Hu et al., 2022) with rank $r = 32$ and scaling factor $\alpha = 64$. LoRA adapters are injected into all query, key, value, and output projection matrices in each transformer layer. During fine-tuning, base model weights are frozen and only the ≈12M LoRA parameters are updated.

5.2. Input Construction

Each training example is constructed as follows. We select a random scene and a random target frame t^* within that scene. The model input is:

$$x = \big[\mathbf{h};\ s_{t^*-3};\ s_{t^*-2};\ s_{t^*-1};\ s_{t^*};\ \texttt{Predict next frame:}\big], \tag{3}$$

where $\mathbf{h}$ is the scene header and semicolons denote concatenation. The target output is s_{t^*+1}. We include $K = 4$ context frames so the model can observe recent velocity trends, which is necessary for accurate collision prediction. Loss is computed only over the output frame tokens.

5.3. Curriculum Training

Inspired by curriculum learning (Bengio et al., 2009; Graves et al., 2017), we train in five stages ordered by scene difficulty. Each stage k trains exclusively on scenes with difficulty $\leq k$ until a validation loss plateau is observed, then advances to the next stage. Concretely:

1. **Stage 1** (diff. 1 only): single-body projectile motion and simple elastic collisions. Establishes baseline Newtonian dynamics.
2. **Stage 2** (diff. ≤ 2): bilateral collisions, ramp traversal, pendulums. Introduces momentum transfer and constraint forces.
3. **Stage 3** (diff. ≤ 3): multi-body stacks, chain constraints, domino cascades. Tests temporal credit assignment over longer causal chains.
4. **Stage 4** (diff. ≤ 4): complex multi-phenomenon scenarios. Trains on the full corpus minus the five hardest scenario types.
5. **Stage 5** (diff. ≤ 5): full training set including the hardest scenarios (plinko, orbit, rube_goldberg, pinball).

The curriculum dataset is built using a parallel scanner (Python multiprocessing with 16 workers) that categorizes 900K files by difficulty in under 20 seconds. Training uses the Unsloth-optimized SFTTrainer (Han & Han, 2024) with packed sequences, bfloat16 precision, and a learning rate of 2×10^{-4} with cosine decay.

5.4. Inference and Rollout

At inference time we use greedy decoding (temperature $T = 0$) with a repetition penalty of 1.08 to discourage degenerate repetition of numeric tokens. The prompt always contains the scene header and the last four frames of context (either ground truth for single-step evaluation, or previously predicted frames for rollout).

For multi-step rollout, we maintain a sliding context window: once the tokenized prompt exceeds 8192 tokens, we drop the oldest frame from the context while always retaining the full scene header. This context management strategy stabilizes long rollouts and prevents prompt-length-induced degradation.

5.5. Browser Deployment via WebGPU

To maximize accessibility, we export the LoRA-merged model to ONNX format and quantize it to q4f16 (4-bit weights, 16-bit activations). The exported model runs in-browser via the @huggingface/transformers.js library with a WebGPU backend. Q4f16 quantization achieves approximately 2× speedup over q4 on WebGPU due to better alignment with 16-bit compute units. This enables server-free physics simulation directly in the browser with no installation requirements.

6. Experiments

6.1. Evaluation Protocol

We evaluate PhysicsLM on the PhysicsScenes validation split (100,020 scenes, 30 scenario types). The OOD subset

(6 held-out types, 20,004 scenes) is reported separately. We assess two complementary aspects:

Single-step prediction. Given four ground-truth context frames, predict the next frame. Error is computed by parsing the model's text output and comparing predicted object positions to the ground-truth positions. Parse success rate measures the fraction of objects whose position/velocity fields are successfully extracted by the parser.

OOD generalization. Same as single-step, but on the six held-out scenario types not seen during training. We further divide OOD scenarios into *near-distribution OOD* (types sharing physical primitives with seen scenarios) and *novel OOD* (types requiring qualitatively new interaction patterns), as they exhibit very different error profiles.

6.2. Metrics

Let $\hat{p}^i$ and p^i denote the predicted and ground-truth position (in pixels) for object i. For single-step evaluation (one predicted frame per scene, N objects) we report:

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}\left\|\hat{p}^i - p^i\right\|_2^2, \qquad \text{RMSE} = \sqrt{\text{MSE}}. \quad (4)$$

All pixel errors are on an 800×600 canvas (diagonal ≈1000 px). Parse rate is the percentage of expected object state lines successfully extracted from the model's text output.

6.3. Points of Comparison

We compute three reference points alongside PhysicsLM (see Appendix D for computation details):

1. **Copy-last:** Predict $\hat{p}^i_{t+1} = p^i_t$ (positions unchanged). Strong for near-static scenes; fails proportionally to object speed.
2. **Linear extrapolation:** Predict $\hat{p}^i_{t+1} = p^i_t + v^i_t \cdot \Delta t$. Achieves ≈0.01 px^2 MSE overall because Pymunk uses semi-implicit Euler ($p_{t+1} = p_t + v_{t+1}\Delta t$), making position updates exactly linear in velocity between collision events. This baseline is therefore near-perfect on most frames by construction; it only fails on the frames where a collision occurs.
3. **LFM2-350M (no fine-tuning):** Base model prompted with our format; produces narrative text, never structured output (parse rate: 0%).

GNS (Sanchez-Gonzalez et al., 2020) and GPT-11M from scratch are deferred to future work (see Section 7).

Table 3. Single-step position RMSE (px) per category on seen scenarios, compared with copy-last baseline (computed on 30 randomly sampled scenes per type). Linear extrapolation achieves ≈0.10 px RMSE across all categories (near-exact by construction; see text). Parse fail rate is 0.0% for PhysicsLM; LFM2 base model achieves 0% parse rate. OOD results: Table 4.

Category	PhysicsLM RMSE ↓	Copy-last RMSE ↓	Loss
Stacking	**0.46**	1.21	0.518
Constraint	**1.01**	3.75	0.572
Collision	**5.16**	7.47	0.688
Ramp	**6.13**	6.20	0.674
Complex	23.64	**4.30**	0.844
Minigame	42.52	**5.77**	0.976
Overall (seen)	18.85	**5.22**	0.660

6.4. Main Results

Table 3 reports per-category position MSE on the PhysicsScenes validation set. The parse failure rate is **0.0%** across all 30 scenario types, including all 6 OOD types—the fine-tuned model always produces well-formed structured output. In contrast, the LFM2-350M base model (without fine-tuning) generates narrative text instead of structured predictions and achieves 0% parse rate, confirming that the physics prediction capability is acquired through fine-tuning, not inherited from the pre-trained backbone.

Performance splits sharply by scenario type. PhysicsLM **outperforms copy-last** on the three structured categories—stacking (2.6× better), constraint (3.7× better), and collision (1.4× better)—and roughly ties on ramp. These gains reflect the model's ability to predict the post-collision velocity from token context, which copy-last cannot do. On complex and minigame scenarios, however, PhysicsLM is 5–7× *worse* than copy-last: the chaotic dynamics of pinball (MSE = 5,910 px^2), avalanche (2,194 px^2), and pong (724 px^2) cause the model's errors to compound, far exceeding a trivial "stand still" prediction. The overall mean MSE is therefore dominated by a small number of catastrophic outliers (median scenario MSE: 1.72 px^2 vs. mean 355 px^2).

The *linear extrapolation* baseline achieves ≈0.10 px RMSE because Pymunk's position update is $p_{t+1} = p_t + v_{t+1}\Delta t$ (semi-implicit Euler): given the velocity at frame t, position is almost exactly predictable without any physics knowledge. The baseline fails only on collision frames, which represent a small fraction of randomly sampled frame pairs. This means single-step MSE alone is not a discriminating metric: even a model that perfectly learned Newtonian integration would achieve near-zero single-step MSE on most frames.

6.5. OOD Generalization

Table 4 breaks down performance on the six held-out scenario types. The results split sharply by how novel the OOD

Table 4. OOD single-step results by held-out scenario type. Near-distribution OOD types share physical primitives with seen scenarios; novel OOD types require qualitatively new interaction patterns absent from training. Parse fail rate is 0.0% for all types.

Scenario	Type	MSE ↓	RMSE ↓
Near-distribution OOD (avg RMSE 0.94 px)			
newtons_cradle	Constraint OOD	0.02	0.14
bowling	Collision OOD	1.51	1.23
ramp_roll	Ramp OOD	2.08	1.44
Novel OOD (avg RMSE 24.79 px, 1.3× worse than seen avg)			
angry_birds	Complex OOD	530.21	23.03
hourglass	Complex OOD	596.34	24.42
pong	Minigame OOD	724.51	26.92

type is relative to the training distribution.

Near-distribution OOD. `newtons_cradle`, `bowling`, and `ramp_roll` all involve physical primitives well-represented in training (elastic chain collisions, ball-lane dynamics, inclined-plane rolls). These types average **0.94 px** RMSE—better than the seen Collision and Ramp categories—because the model successfully applies patterns learned from structurally similar training scenarios. `newtons_cradle` in particular achieves MSE = 0.02 px^2, the best of any evaluated scenario, because the model has already internalized elastic chain-collision dynamics from seen constraint scenarios (`chain`, `pendulum`).

Novel OOD. `angry_birds`, `hourglass`, and `pong` require qualitatively new interaction patterns that have no direct analogue in the training set. These average **24.79 px** RMSE—1.3× *worse* than the seen-scenario average of 18.85 px. `pong` (MSE = 724 px^2) requires predicting ball reversal at paddle contact; `hourglass` (MSE = 596 px^2) involves dense granular flow through a narrow neck with many simultaneous micro-collisions; `angry_birds` (MSE = 530 px^2) presents multi-stage destruction chains where positional error at the initial impact cascades across all downstream blocks.

Despite these failures, the parse rate remains **100%** even on novel OOD types, confirming that format generalization is robust even when physics reasoning degrades.

6.6. Per-Scenario Breakdown

The ten best and ten worst scenarios by MSE span nearly five orders of magnitude (0.02–5910 px^2), illustrating that scenario type is the dominant predictor of PhysicsLM accuracy. The five best scenarios are all constraint- or stacking-type (`newtons_cradle`, `bridge`, `jenga`, `dominos`, `pyramid`), while the five worst are all high-entropy collision-dominated scenarios (`pinball`, `avalanche`, `pong`, `hourglass`, `angry_birds`). Cross-entropy loss is a poor proxy for position MSE: `plinko` achieves very low loss (0.282) because the model learns the *format* of the output well, yet has high positional error (MSE = 132 px^2) because the scenario involves chaotic ball paths that are hard to predict precisely.

7. Discussion and Limitations

Collision Outliers. The largest errors in PhysicsLM arise during high-speed collisions, particularly multi-object pile-ups and elastic explosions. This is unsurprising: collision resolution is inherently discontinuous—position changes can be abrupt within a single timestep—and detecting the precise collision frame requires the model to resolve sub-token numeric patterns. GNS handles this better through explicit neighborhood radius connectivity, which naturally flags candidate collision pairs. Future work could add collision-frame flags to the text encoding (e.g., `Frame 42: collision detected.`) to give the LM a softer signal.

Numerical Precision in Text. We use 4-decimal-place encoding (≈ 0.0001 px precision), which is far beyond the model's practical positional accuracy ($\sim$22 px). In principle, a more compact encoding (e.g., integers rounded to 0.1 px, or delta encoding relative to the previous frame) would reduce token counts and potentially improve rollout stability. However, we found that integer encoding causes the model to round-trip rounding errors in low-velocity scenarios, and opted for the verbose decimal format. This is a clear efficiency-accuracy tradeoff for future work.

3D Extension. The current system is limited to 2D rigid bodies. Extending to 3D (e.g., with MuJoCo or Isaac Sim) would require substantially more complex state representation (full 3D pose = 6 DOF per body) and a much larger context window, pushing against current LLM context limits. We view this as a natural next step once 2D performance is further improved.

Energy Non-Conservation. Unlike Hamiltonian Neural Networks (Greydanus et al., 2019), PhysicsLM does not enforce energy conservation. Over long rollouts, we observe a slow drift toward energy decrease (inelastic rounding in velocity predictions). Regularizing the model toward energy-conserving predictions is an open problem for LM-based physics.

Dataset Coverage. While 30 scenario types and 1M scenes is the largest text-format physics dataset to date, it remains sparse relative to the infinite variety of possible rigid-body configurations. The model has not been exposed to fluid dynamics, deformable bodies, or aerodynamic forces. Expanding the dataset generator and increasing scene diver-

sity are immediate future directions.

GPT-from-Scratch Baseline (Negative Result). We trained an 11M-parameter GPT-style transformer from scratch on the same PhysicsScenes corpus using a digit-level tokenizer and muP hyperparameter scaling. Training was halted after 500 gradient steps (compared to 7,815 steps for PhysicsLM) due to resource constraints. At this checkpoint the model produces syntactically partial frames with diverging coordinate values, making quantitative MSE comparison impossible. We report this as a *negative result*: small transformers trained from scratch on this task require substantially more compute to converge than fine-tuning a pre-trained 350M-parameter language model. The pre-trained model's numerical and structural priors (decimal formatting, positional reasoning, symbol copying) appear to provide a strong inductive bias that dramatically reduces sample complexity for physics state modeling. Fully converging the scratch baseline is left to future work with dedicated compute.

Scaling Analysis. The most immediate open question is whether performance scales predictably with model size and data volume. Smaller pre-trained LMs (<100M parameters) may lack sufficient capacity to capture multi-body collision geometry, while LMs at the 1B–7B scale should have stronger numerical reasoning but come with much higher inference cost. Based on our digit-level tokenizer analysis, each 200-frame scene uses ≈4,200 tokens; a 7B model with a 32K context window could in principle fit the entire rollout in one forward pass, enabling *full-trajectory* attention rather than the sliding-window approximation used here. On the data side, our 900K training scenes cover 24 scenario types; extrapolating the category-level MSE relationship to 100× more data would require constructing a self-supervised curriculum, since manual difficulty labels do not scale. Investigating whether neural-scaling power laws (Kaplan et al., 2020) hold for physics text modeling is a promising research direction.

8. Conclusion

We have presented PhysicsLM, a system that reframes 2D rigid-body physics simulation as autoregressive language modeling. By encoding simulation state as structured decimal text and fine-tuning a 350M-parameter LFM2 model via LoRA on 900K generated scenes with a 5-stage difficulty curriculum, we achieve **0.0% parse failure rate** across all 30 scenario types (including 6 held-out OOD types), an overall position RMSE of **18.85 px** on seen scenarios, and excellent OOD generalization on constraint and collision types (e.g., `newtons_cradle` RMSE = 0.14 px despite not appearing in training). Performance degrades on high-entropy scenarios (`pinball`, `avalanche`, `pong`), pointing to the limits of token-level next-frame prediction for chaotic dynamics. The PhysicsScenes dataset, training code, and WebGPU browser demo are publicly released. We hope this work spurs further exploration of LM-based physical world modeling, including energy-conserving decoding, 3D extension, and richer OOD benchmarks.

Acknowledgments

We thank the Liquid AI team for making LFM2-350M publicly available, Viktor Blomqvist for maintaining Pymunk, and the HuggingFace team for the Transformers library and WebGPU inference infrastructure.

References

Battaglia, P. W., Pascanu, R., Lai, M., Jimenez Rezende, D., and Kavukcuoglu, K. Interaction networks for learning about objects, relations and physics. In *Advances in Neural Information Processing Systems*, volume 29, 2016.

Bengio, Y., Louradour, J., Collobert, R., and Weston, J. Curriculum learning. *Proceedings of the 26th International Conference on Machine Learning*, pp. 41–48, 2009.

Blomqvist, V. Pymunk: A python 2d physics library, 2024. URL http://www.pymunk.org/.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33: 1877–1901, 2020.

Chen, L., Lu, K., Rajeswaran, A., Lee, K., Grover, A., Laskin, M., Abbeel, P., Srinivas, A., and Mordatch, I. Decision transformer: Reinforcement learning via sequence modeling. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

Cranmer, M., Greydanus, S., Hoyer, S., Battaglia, P., Spergel, D., and Ho, S. Lagrangian neural networks. *arXiv preprint arXiv:2003.04630*, 2020.

Graves, A., Bellemare, M. G., Menick, J., Munos, R., and Kavukcuoglu, K. Automated curriculum learning for neural networks. In *International Conference on Machine Learning*, pp. 1311–1320, 2017.

Greydanus, S., Dzamba, M., and Yosinski, J. Learned physics: Lagrangian and hamiltonian dynamics from data. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

Ha, D. and Schmidhuber, J. World models. *arXiv preprint arXiv:1803.10122*, 2018.

Hafner, D., Lillicrap, T., Ba, J., and Norouzi, M. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020.

Han, D. and Han, M. Unsloth: Efficient fine-tuning of large language models. In *arXiv preprint arXiv:2407.10490*, 2024.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

Kipf, T., Fetaya, E., Wang, K.-C., Welling, M., and Zemel, R. Neural relational inference for interacting systems. In *International Conference on Machine Learning*, pp. 2688–2697, 2018.

Li, Y., Wu, J., Tedrake, R., Tenenbaum, J. B., and Torralba, A. Learning particle dynamics for manipulating rigid bodies, deformable objects, and fluids. In *International Conference on Learning Representations*, 2019.

Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., and Anandkumar, A. Fourier neural operator for parametric partial differential equations. *arXiv preprint arXiv:2010.08895*, 2021.

Liquid AI. LFM-2: Scalable language foundation models for the edge. *Technical Report, Liquid AI*, 2024.

OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

Pfaff, T., Fortunato, M., Sanchez-Gonzalez, A., and Battaglia, P. W. Learning mesh-based simulation with graph networks. In *International Conference on Learning Representations*, 2021.

Sanchez-Gonzalez, A., Godwin, J., Pfaff, T., Ying, R., Leskovec, J., and Battaglia, P. W. Learning to simulate complex physics with graph networks. In *International Conference on Machine Learning*, pp. 8459–8468. PMLR, 2020.

Satorras, V. G., Hoogeboom, E., and Welling, M. E(n) equivariant graph neural networks. In *International Conference on Machine Learning*, pp. 9323–9332. PMLR, 2021.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., and Zhou, D. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

Yang, S., Nachum, O., Du, Y., Wei, J., Abbeel, P., and Schuurmans, D. Foundation models for decision making: Problems, methods, and opportunities. In *arXiv preprint arXiv:2303.04129*, 2023.

Yang, Z., Liu, Q., Chen, L., Liu, Q., Zhao, J., Qin, L., and Che, W. Can large language models understand physics? benchmarking LLMs on physical reasoning. *arXiv preprint arXiv:2405.04814*, 2024.

A. Text Encoding Format

Each simulation scene is serialized as a JSONL file: a single header line followed by $T = 200$ frame lines. At training and inference time the JSONL is converted to a plain-text prompt. A scene header encodes static scene properties:

```
Scene: billiards (seed=5000042)
Gravity: (0.0000, -981.0000)  Timestep: 0.016667
Objects: 11 (circles)
obj_0: mass=1.00 friction=0.05 elasticity=0.92 r=15.0
... (one line per object)
```

Each frame is encoded as:

```
Frame 42: Objects in motion.
  obj_0: pos=(312.4800, 187.2300),
         vel=(45.2100, -123.4500)
  obj_1: pos=(201.1200, 98.7700),
         vel=(-12.3400, 34.5600),
         a=0.8700, av=1.2300
```

Angle (`a`) and angular velocity (`av`) are included only when their absolute value exceeds 0.001 radians (or rad/s). All numeric values use 4 decimal places, corresponding to ≈0.0001 px/px·s precision. A 5-object scene averages ≈120 tokens per frame; a full 200-frame scene encodes as ≈25K tokens, which exceeds LFM2's 8192-token context.

Token budget at training time. Training examples are constructed as (4-frame context → 1-frame target). For a 5-object scene, each example uses ≈600 tokens for context and ≈120 tokens for the target. SFTTrainer packs multiple such examples per 2048-token sequence (roughly 2–3 examples per sequence). With batch size 4 and 7,815 gradient steps, the model trains on approximately 250K distinct (context, target) frame-pairs from the 900K-scene corpus, covering ≈0.14% of all available pairs per epoch. This is a

small data fraction; training for more epochs or more steps is an immediate avenue for improvement.

B. Training Hyperparameters

Table 5. PhysicsLM training configuration.

Hyperparameter	Value
Base model	LFM2-350M
Fine-tuning method	LoRA (r=32, α=64)
LoRA target modules	Q, K, V, O projections (all layers)
Trainable parameters	≈12M (of 350M total)
Context window	8192 tokens
Training context frames	$K = 4$
Max sequence length	2048 tokens (packed)
Batch size	4
Gradient accumulation	1
Optimizer	AdamW (β_1=0.9, β_2=0.999)
Learning rate	2×10^{-4} with cosine decay
Warmup steps	100
Total gradient steps	7,815 (across all 5 curriculum stages)
Precision	bfloat16
GPU	NVIDIA RTX A6000 (44.4 GB)
Training time	≈18 hours
Dataset generation	
Simulator	Pymunk / Chipmunk2D, 60 FPS
CPU cores	22
1M-scene generation time	≈29 minutes
Uncompressed size	≈582 GB

Generation speed plausibility. Each of the 1M scenes runs 200 Pymunk simulation steps on an average of ≈5 objects. At 22 cores, this is approximately 45,000 scenes per core, each taking ≈0.039 seconds. A single-core Pymunk step for 5 objects runs in $\sim$10 μs, giving 200 steps $\times$ 5 objects $\times$ 10 μs = 10 ms per scene, plus serialization overhead; 39 ms total is consistent with this estimate.

Training steps plausibility. 7,815 gradient steps at batch size 4 with packed 2048-token sequences corresponds to ≈250K frame-pair training examples. The curriculum proceeds through 5 stages; early stages (difficulty 1–2) converge faster and require fewer steps, while later stages (difficulty 4–5) require more. Loss curves are monotonically decreasing within each stage; no divergence was observed.

C. WebGPU Export and Quantization

The LoRA-merged model is exported to ONNX using the HuggingFace Optimum library. We use q4f16 quantization: weights are stored in 4-bit but activations (including KV cache) remain in float16. Two Cast nodes in the ONNX graph (`/model/gqa_attention_bias/Cast` and `/model/shared_conv/Mask_Cast`) output float32 by design; we patch these by rewiring their output tensors to float16 using a custom post-processing script, eliminating the type mismatch that otherwise prevents WebGPU inference. The fixed model runs at ≈2× the speed of q4-only quantization on WebGPU hardware.

D. Baseline Computation Details

Copy-last and linear extrapolation baselines are computed by generating 30 fresh scenes per scenario type (seeds 5,000,000 to 5,000,029) using the same Pymunk simulator. For each scene, 5 random frame-pairs (s_t, s_{t+1}) are sampled uniformly from frames 4–198. Copy-last error is $\|\hat{p}^i_{t+1} - p^i_{t+1}\|^2$ with $\hat{p}^i_{t+1} = p^i_t$. Linear error uses $\hat{p}^i_{t+1} = p^i_t + v^i_t \Delta t$. PhysicsLM numbers are from the stage-0 evaluation (teacher-forced single-step mode) on a separate held-out evaluation set; they are not directly comparable object-for-object but are from the same scenario-type distribution.

Note that linear extrapolation achieves ≈0.01 px^2 MSE because Pymunk uses semi-implicit Euler: $v_{t+1} = v_t + a\Delta t$, $p_{t+1} = p_t + v_{t+1}\Delta t$. Given v_t (available as input), the true p_{t+1} differs from the linear prediction $p_t + v_t\Delta t$ only by $a\Delta t^2 = g(1/60)^2 \approx 0.27$ px (in the gravity direction). For horizontal motion or near-rest objects, the error is zero. This means linear extrapolation is effectively an oracle on non-collision frames; its failure mode (incorrect velocity prediction after collision) is the same gap that PhysicsLM is designed to close.